

Aerostructural Trade Theory

Braxton Lopez

02 March 2026

1 Vocabulary

Structural Damping Like almost all physical objects, aerodynamic structures will experience damping effects. When a force is applied to a wing/fuselage, oscillations will occur, which will eventually stabilize out as the excess energy is dissipated through internal frictional forces (throughout the entirety of the structure and especially at the joints), aerodynamic damping, etc. This energy dissipation is known as damping, and structural damping is a product of only the damping applied by internal forces. There are two main types of structural damping: 1) viscous damping, which is proportional to velocity and 2) hysteretic damping, which is a function of the material's property. Without these and other forms of damping, an ideal structure would continue to oscillate forever after a force is applied. For this reason, damping is required for aerodynamic structures in order to increase passenger comfort/ride quality. You can visualize structural damping by imagining a bouncy ball that will bounce decreasingly high with every bounce, despite the minimal effect of air drag. This is because of structural damping, despite it being on a much smaller scale than an aerodynamic structure.

Aerodynamic Damping Unlike structural damping, aerodynamic damping comes from external forces acting on the structure. In this case, it would be the air in the freestream that exerts this force. As a wing moves, vibrates, or otherwise oscillates, the aerodynamic loading changes. The freestream will either oppose or assist this movement, depending on the conditions. Due to the unsteady movement's of the structure/freestream, the direction of the damping forces applied can vary from being negative (adding energy and increasing vibrations) to positive (dissipating energy from the structure into the air). The magnitude of this force is directly proportional to the freestream velocity. When aerodynamic damping becomes sufficiently negative to cancel structural damping, flutter can occur. Flutter is when a wing oscillates increasingly and uncontrollably faster, causing passenger discomfort and a high probability of wing damage (in many cases the wing will tear itself away from the aircraft).

Static Stability and Instability Static stability is an aerodynamic structure's tendency to return to its original equilibrium position after a disturbance. This describes the structure immediately after the disturbance and is a function of geometry, stiffness, and the locations of the center of gravity and the aerodynamic center. It is not a function of damping as it does not describe how the structure reacts over time to these displacements/disturbances, it only describes how the structure reacts *immediately* afterwards. There are three type of static stability: 1) Longitudinal, 2) Lateral, and 3) Directional static stability. Longitudinal stability is the pitch stability, and instability in this direction will cause the airplane to roll. Lateral instability will cause the airplane's nose to tip either upwards or downwards. Directional instability will cause the nose to move to the right or left.

In order to ensure static stability, placing the center of gravity ahead of the aerodynamic

center is critical. For more information on how to determine the aerodynamic center, see this footnote¹.

Static instability is highly dangerous when flying aircraft. Static instability occurs when the plane's first reaction to any disturbance is to continue moving in the direction the disturbance's force is applied instead of returning to its original position. This tendency can rapidly result in loss of control for the pilot.

Geometric Nonlinearity In *iii. Matrix Formulation for Two-Node Beam Elements* of the document *497RWeeks7-8*, we discussed how matrix formulation can be used to model linear effects, specifically the relationship $\{F\} = [K_e]\{d\}$. This function is based on the assumption that an element has two nodes and all properties in the space between those two nodes follow a linear relationship. When we have a nonlinear effect (such as the stiffness increasing as deformation is increased), this relationship no longer holds true. Because these properties change, the equilibrium equations also change as the structure deforms, therefore you aren't solving the same structure you began with. The relationship between force and displacement is no longer proportional.

Geometric nonlinearity occurs when, under deformation, the geometry of a structure changes sufficiently for these changes to have to be included in an analysis. An example of this is necking during a tensile test of an aluminum dog-bone specimen. In the attached video², the deformation is uniform until the specimen begins necking, during which the cross-sectional area begins to significantly change around the middle of the specimen. This causes increased stresses, leading to increased deformation, etc., causing a nonlinear change in the equilibrium balance equations. As the total deformation increases, there's an increased likelihood of having to account for geometric nonlinearity. A linear analysis will be increasingly inaccurate as the displacement of a beam/structure is increased.

P- Δ Effect For a full breakdown of the causes behind the P- Δ , as well as the derivation of the relationship $P - \Delta$, see this footnote³.

The $P - \Delta$ effect is nonlinear, where P is the applied force and Δ is the displacement. Note that this effect is only true while P is a compressive force. Most isotropic beams under compressive forces will experience a mostly linear stress concentration throughout the beam; however, as a long, slender beam experiences these forces, bending/buckling begins to occur. As this happens, there begins to be a moment arm that increases the stress concentration along certain areas of the beam. As the deflection increases, so does the moment arm, which causes greater stresses, causing more deflection, and this cycle can continue until failure. This effect is nonlinear, with deformation increasing as deflection increases. Note that as the deflection increases, the geometry of the beam also changes and this must be included in any calculations/simulations. In other words, as the beam

¹A. Ning, 2018. <https://scholarsarchive.byu.edu/cgi/viewcontent.cgi?article=1027&context=books>, pg 95-96

²<https://www.youtube.com/watch?v=Pq3KRCf4Q9Ut=2s>

³<https://www.youtube.com/watch?v=IKm9FIUq4cst=392s>

continues to bend, its stiffness decreases. You can see this in columns that have bent, airplane wings that have a dihedral angle $\phi \neq 0^\circ$ (compressive forces are being applied on the wing due to lift), rocket fins, etc.

Added Mass As a body passes through any fluid, it displaces the fluid through which it passes, and part of the body's energy is lost in accelerating the fluid. The body must overcome the inertia of the fluid through which it passes as well as its own inertia. Just as a point mass, block mass, etc., a fluid has inertia that must be overcome in order to accelerate it. This extra resistance applied to the body by the forces required to accelerate the fluid is called the "added mass", and effectively behaves as if there were simply an increased mass of the body:

$$m_{effective} = m_{body} + m_{added} \quad (1)$$

The added mass is a function of the fluid density, the body's shape, the direction of motion, and the boundary conditions. For example, the added mass of a sphere is:

$$m_{added} = \frac{1}{2}\rho V \quad (2)$$

where ρ is the fluid density and V is the volume of the sphere. For an aerodynamic structure, the added mass is significantly more complicated to compute, and also varies depending on if the airflow is compressible or incompressible. The added mass also increases as the fluid's density increases.

A body with added mass will accelerate more slowly due to having to overcome greater inertia. Its greater inertia requires more force to maneuver, and also decreases the natural frequency. It will also alter the stability due to the load's (the load generated by the fluid's inertia) distribution across the surface of the body. Without added mass, maneuvering requires less effort/time, the body has increased acceleration, and the body has a higher natural frequency. Added mass is not drag, as drag dissipates energy proportional to the free stream's velocity. Added mass is a term used to model how a body will react to changes in applied forces (for example, if an airplane wants to turn, it must move the air around the wings up/down in order to turn). Added mass opposes acceleration and drag opposes velocity.

2 Theory

2.1 Structural Dynamics Fundamentals

i. Structural Damping Like almost all physical objects, aerodynamic structures will experience damping effects. When a force is applied to a wing/fuselage, oscillations will occur, which will eventually stabilize out as the excess energy is dissipated through internal frictional forces (throughout the entirety of the structure and especially at the joints), aerodynamic damping, etc. This energy dissipation is known as damping, and structural

damping is a product of only the damping applied by internal forces. There are two main types of structural damping: 1) viscous damping, which is proportional to velocity and 2) hysteretic damping, which is a function of the material's property. Without these and other forms of damping, an ideal structure would continue to oscillate forever after a force is applied. For this reason, damping is required for aerodynamic structures in order to increase passenger comfort/ride quality. You can visualize structural damping by imagining a bouncy ball that will bounce decreasingly high with every bounce, despite the minimal air drag. This is because of structural damping, despite it being on a much smaller scale than an aerodynamic structure. A mass-spring damper system will have any induced oscillations dampened out and equalized; however, in this case, the damping forces are applied externally, whereas structural damping is purely internal (a mass-spring system requires a physical damper to be connected for damping to occur in an ideal system; however, structural damping happens without any external damping forces needing to be applied).

ii. P- Δ Effect For a full breakdown of the causes behind the P- Δ , as well as the derivation of the relationship $P - \Delta$, see this footnote⁴.

The $P - \Delta$ effect is nonlinear, where P is the applied force and Δ is the displacement. Note that this effect is only true while P is a compressive force. Most isotropic beams under compressive forces will experience a mostly linear stress concentration throughout the beam; however, as a long, slender beam experiences these forces, bending/buckling begins to occur. As this happens, there begins to be a moment arm that increases the stress concentration along certain areas of the beam. As the deflection increases, so does the moment arm, which causes greater stresses, causing more deflection, and this cycle can continue until failure. This effect is nonlinear, with deformation increasing as deflection increases. Note that as the deflection increases, the geometry of the beam also changes and this must be included in any calculations/simulations. In other words, as the beam continues to bend, its stiffness decreases. You can see this in columns that have bent, airplane wings that have a dihedral angle $\phi \neq 0^\circ$ (compressive forces are being applied on the wing due to lift), rocket fins, etc.

iii. Geometry and Resonance Frequency A wing's geometry can significantly affect its resonance frequency f_r . Geometry will affect every property, some directly and some indirectly. The geometry will affect the total stiffness of a beam, the total mass, the stresses, the moments of inertia, the structural damping, etc., which can all affect the resonance frequency. The resonance frequency is proportional to:

$$f_r \propto \sqrt{\frac{EI}{\rho AL^4}} \quad (3)$$

where A is the planar cross-sectional area, ρ is the material's density, E is the modulus of elasticity, I is the moment of inertia, and L is the length. We can see from this that length

⁴<https://www.youtube.com/watch?v=IKm9FIUq4cst=392s>

plays the largest factor in decreasing the frequency, and doubling the length will decrease the frequency by a factor of 4. Increasing the width or height of the beam will increase the moment of inertia, subsequently increasing the resonance frequency. There are many ways to change the geometry to alter a beam/wing's resonance frequency.

There are several advantages to increasing the f_r of a wing. Having a stiffer wing makes modeling the aerodynamic effects significantly easier, as it will minimize deflections based on the freestream's unsteady flow nature (such as what happens when flying over a mountain or in a storm). Stiffer wings (higher f_r) also allow for increased control, as you get more immediate responses from a plane. Increasing the f_r will also cause the flutter speed to occur at higher airspeeds, and this is the principle reason for increasing f_r . For a full list of the effects of flutter, see *vi. Flutter*; however, this is extremely dangerous and increasing the speed at which flutter occurs simply means the plane can fly faster and fly safer.

There are; however, several disadvantages to increasing the f_r of a wing. Increased stiffness of an beam implies increased weight (if using the same material), and the plane will require more lift in order to carry the same weight (or the allowable payload weight must be decreased). This also decreases the plane's range, climb rate, and maneuverability. Stiffer wings are also more expensive to manufacture as they require more material. Having increased control from stiffer wings also worsens the effects that unsteady airflows can have on the plane (such as what happens when flying over a mountain or in a storm). There are pros and cons to increasing the f_r of a wing, and you must carefully balance them.

2.2 Aerodynamic Response and Stability

iv. Aerodynamic Damping and Stability Derivative Unlike structural damping, aerodynamic damping comes from external forces acting on the structure. In this case, it would be the air in the freestream that exerts this force. As a wing moves, vibrates, or otherwise oscillates, the aerodynamic loading changes. The freestream will either oppose or assist this movement, depending on the conditions. Due to the unsteady movement's of the structure/freestream, the direction of the damping forces applied can vary from being negative (adding energy and increasing vibrations) to positive (dissipating energy from the structure into the air). The magnitude of this force is directly proportional to the freestream velocity. When aerodynamic damping becomes sufficiently negative to cancel structural damping, flutter can occur. Flutter is when a wing oscillates increasingly and uncontrollably faster, causing passenger discomfort and a high probability of wing damage (in many cases the wing will tear itself away from the aircraft). Aerodynamic damping is not caused by drag. Drag can only remove energy from structures, while damping can add or remove energy depending its phase.

There are different governing sources of aerodynamic damping. At low angles of attack, induced flow is the primary source, at high angles of attack it's wake dynamics, and during high speed flight it's compressibility changes in the freestream's fluid.

Stability derivatives quantify how the aerodynamic forces/moments change in response to the aircraft's motion in relation to the freestream. `VortexLattice.jl` uses stability deriva-

tives to find the aerodynamic damping, as aerodynamic damping occurs when these forces or moments oppose the aircraft's movements. VortexLattice finds these derivatives through:

Listing 1: Stability Derivatives

```

1 dCFs, dCMs = stability_derivatives(system)
2 # traditional names for each stability derivative
3 CDa, CYa, CLa = dCFs.alpha
4 Cla, Cma, Cna = dCMs.alpha
5 CDb, CYb, CLb = dCFs.beta
6 Clb, Cmb, Cnb = dCMs.beta
7 CDp, CYp, CLp = dCFs.p
8 Clp, Cmp, Cnp = dCMs.p
9 CDq, CYq, CLq = dCFs.q
10 Clq, Cm q, Cnq = dCMs.q
11 CDr, CYr, CLr = dCFs.r
12 Clr, Cmr, Cnr = dCMs.r

```

Aerodynamic damping occurs when any of these coefficients is negative (opposing the aircraft's movements).

2.3 Aero-Structural Coupling

v. Deflection's Effect on a Wing's Aerodynamic Response If a wing experiences deflection from lift, it will cause the wing to move upwards, effectively causing an increase in the dihedral angle ϕ (in relation to the fuselage). This can introduce an axial force that pushes into the fuselage. As discussed in *Geometric Nonlinearity* and *P- Δ Effect*, as a beam (or wing) experiences large deflections, its geometry will change, and this will affect almost all the properties of a wing's response, as this can change the shape of the airfoil. This; however, is generally only a concern under unusually large deflection. The way a wing deflects also influences the response. For example, if a wing twists nose-up, then the local angle of attack is increased, causing more lift to be produced. Similarly, if it twists nose-down, then the local angle of attack is decreased, causing a decrease in lift. Also note that deflection causes a change in the stability derivatives and the aerodynamic damping, and these changes increase as deflection is increased. Note that all of these changes are not independent. For example, if a wing twists, causing more lift to be locally produced, it will affect the effective angle of attack, which can affect the aerodynamic damping, etc., until the wing's aerodynamic response is generally different than before deformation occurred. For this reason, it's important to understand how a wing behaves during periods of low lift (or deformation) and high lift.

2.4 Aeroelastic Instability

vi. Flutter Flutter is the result of an unstable system. The following is an explanation of what causes flutter, and can mostly be found in *Static Stability and Instability* of this

document.

As a wing moves, vibrates, or otherwise oscillates, the aerodynamic loading changes. The freestream will either oppose or assist this movement, depending on the conditions. Due to the unsteady movement's of the structure/freestream, the direction of the aerodynamic damping forces applied can vary from being negative (adding energy and increasing vibrations) to positive (dissipating energy from the structure into the air). The magnitude of this force is directly proportional to the freestream velocity. When aerodynamic damping becomes sufficiently negative to cancel structural damping, flutter can occur. Flutter is when a wing oscillates increasingly and uncontrollably faster, causing passenger discomfort and a high probability of wing damage (in many cases the wing will tear itself away from the aircraft). Drag can only remove energy from structures, while damping can add or remove energy depending on its phase, and flutter is when the aerodynamic damping forces are adding energy to the system instead of removing it.

2.5 Fluid-Structure Interaction Effects

vii. Added Mass As a body passes through any fluid, it displaces the fluid through which it passes, and part of the body's energy is lost in accelerating the fluid. The body must overcome the inertia of the fluid through which it passes as well as its own inertia. Just as a point mass, block mass, etc., a fluid has inertia that must be overcome in order to accelerate it. This extra resistance applied to the body by the forces required to accelerate the fluid is called the "added mass", and effectively behaves as if there were simply an increased mass of the body:

$$m_{effective} = m_{body} + m_{added} \quad (4)$$

The added mass is a function of the fluid density, the body's shape, the direction of motion, and the boundary conditions. For example, the added mass of a sphere is:

$$m_{added} = \frac{1}{2}\rho V \quad (5)$$

where ρ is the fluid density and V is the volume of the sphere. For an aerodynamic structure, the added mass is significantly more complicated to compute, and also varies depending on if the airflow is compressible or incompressible. The added mass also increases as the fluid's density increases.

A body with added mass will accelerate more slowly due to having to overcome greater inertia. Its greater inertia requires more force to maneuver, and also decreases the natural frequency. It will also alter the stability due to the load's (the load generated by the fluid's inertia) distribution across the surface of the body. Without added mass, maneuvering requires less effort/time, the body has increased acceleration, and the body has a higher natural frequency. Added mass is not drag, as drag dissipates energy proportional to the free stream's velocity. Added mass is a term used to model how a body will react to changes in applied forces (for example, if an airplane wants to turn, it must move the air around the wings up/down in order to turn). Added mass opposes acceleration and drag opposes velocity.

3 Application

i. Bending Moments at Root

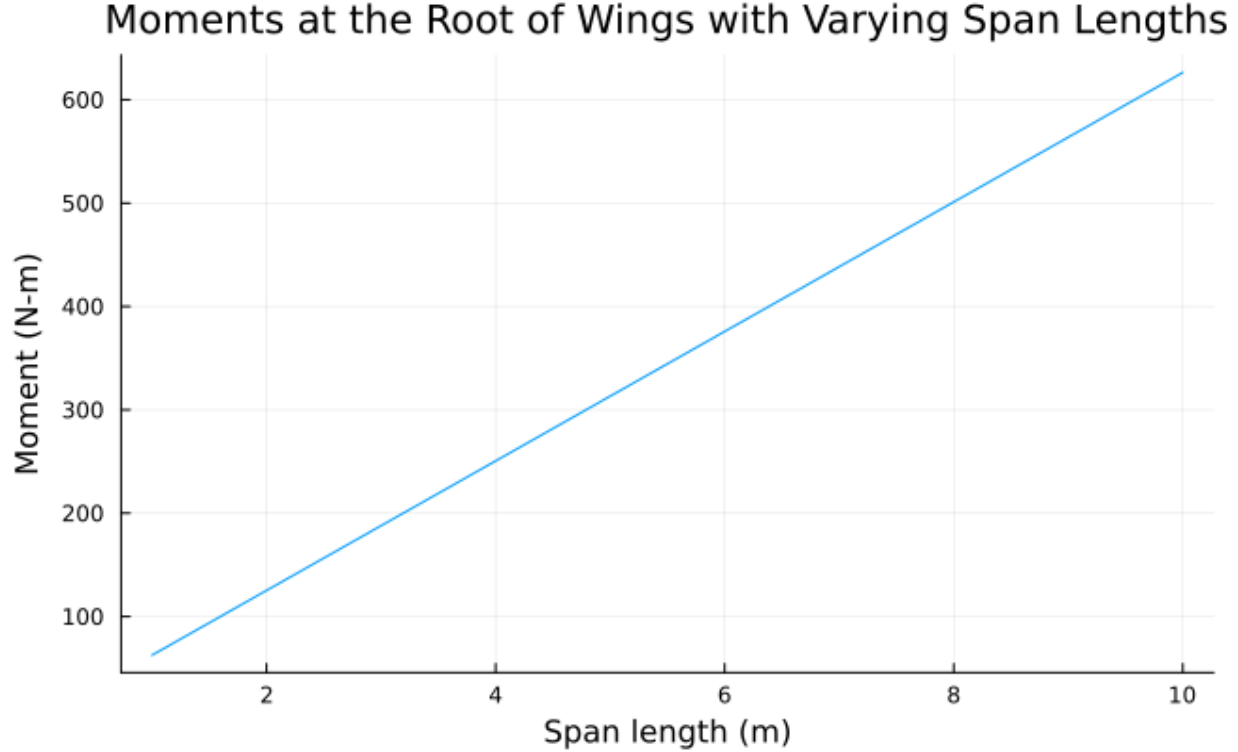


Figure 1: Details how the moment at the root of a wing will vary with the span length.

The following are the results of *3a i.* demonstrating how the bending moment at the root of a half-wing scales with the span, found using the analytical equation of the circulation from an elliptical wing, as shown:

$$\Gamma(y) = \Gamma_0 \sqrt{1 - \left(\frac{2y}{b}\right)^2} \quad (6)$$

where $\Gamma_0 = 1$. We are finding the relationship between the moment at the root and the span length b , and do not need an accurate value of Γ_0 (therefore $\Gamma_0 = 0$, and the freestream velocity is 10m/s. As you can see, as the span length increases, the bending moment at the root increases in a linear pattern. This same relationship holds true for large span lengths (above 10m, which are uncommon/unpractical in real life). This relationship follows the logic that an increased span length allows for more lift to be produced, and the increased length allows for a greater moment to be produced (larger moment arms). The following code was used to produce these results:

Listing 2: Simulation Code

```

1 #FINDING THE EFFECT THE SPAN LENGTH OF THE WING HAS ON THE
2 #MOMENT AT IT'S ROOT
3 using FLOWMath
4 using Plots
5 moments_analytical = []
6 x_values_ma = []
7 density = 0.38
8 Vinf = 10
9 for b in range(1, 10, length=10)
10     push!(x_values_ma, b)
11     moment_wing = []
12     for y in range(0, b/2, length=100)
13         c = 1 * sqrt(1 - (2*y/(b))^2)
14         lftpus = density * Vinf * c # converting circlation to
            lift per unit span
15         added_moment = lftpus * y
16         push!(moment_wing, added_moment)
17     end
18     total_moment = sum(moment_wing)
19     push!(moments_analytical, total_moment)
20 end
21 plot(x_values_ma, moments_analytical, xlabel="Span_length(m)",
        ylabel="Moment",
22 title="Moments_at_the_Root_of_Wings_with_Varying_Span_Lengths",
23 size=(650, 400), legend=false)

```

ii. Trade Study

In this trade study, three different wing geometries were analyzed: an elliptical wing, a tapered wing, and a rectangular wing, with the specific geometries specified below in the code found in Appendix A. The material used was steel 1018. The following are the results of the vertical displacements, rotation about the y-axis, the moments and shear forces, and the lift distributions. VortexLattice.jl was used in order to find the lift distributions, and GXBeam.jl was used to find the rest of the information, noting that any changes in geometry through the length of the wing were accounted for (see Appendix A for more details).

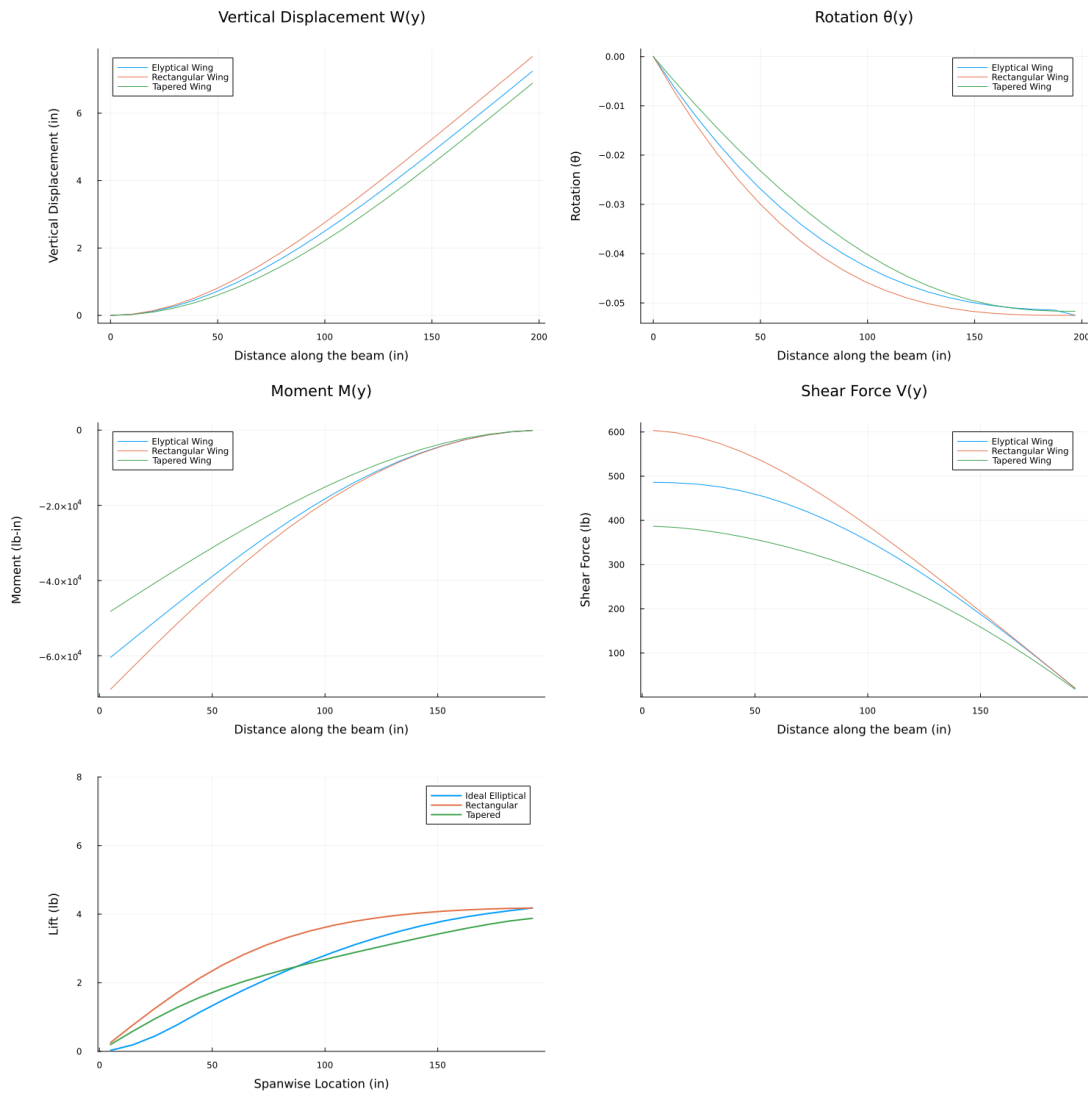


Figure 2: Results of the trade study, with the specific values listed below and in Appendix A.

In this analysis, the length of the wings are 397.7in (the half wings are 196.85in), the density of the freestream air is $0.38kg/m^3$, the chords at the root of the wings are all 39.3701in, and the freestream velocity was $328ft/s$. For more information on the specific geometry/conditions, see Appendix A. The induced drag coefficient of the entire wing and the total mass of each wing were also found, as shown below. The price estimate to manufacture each wing depends on the processes used and is not listed here.

Wing Shape	Mass (lbs)	Total Induced Drag Coefficient
Elliptical	1691	0.0022778
Rectangular	2201	0.0029706
Tapered	1321	0.00134556

Table 1: Mass and total drag coefficient (for the entire wing) for each wing geometry.

As we can see, a wing's geometry can significantly affect its properties/performance. The following test was performed using a freestream velocity of $328ft/s$ (224mph), and these differences will dramatically increase as the velocity increases.

Appendix A - Code

Listing 3: Simulation Code

```
1 using FLOWMath
2 using LinearAlgebra
3 using Plots
4 using VortexLattice
5 using GXBeam
6 using Interpolations
7 using Measures
8
9 ###TRADE STUDY USING THE AN ELYPTICAL WING, RECTANGULAR WING, AND
  A TAPERED CHORD WING
10
11 ###Elyptical Wing
12 bref = 10 # reference span
13 n = 20      # number of spanwise panels
14 xle = zeros(n)      # no sweep
15     = range(0,      /2; length=n)
16 b_half = bref*0.5
17 yle = range(0, b_half; length=n)
18 zle = zeros(n)
19 c_root = 1
20 chord = c_root * sqrt.(1 .- (yle ./ b_half).^2)
21 theta = zeros(n)
22 phi    = zeros(n)
23 fc = fill((xc) -> 0.0, n)
24 # camberline function for each section (y/c = f(x/c))
25 #This function describes the camberline of the airfoil sections
  along the span of the wing, creating the upper and
26 #lower edges of the airfoils. This function is set to zero (the
  first number), indicating that the wing has no camber
27 #and is a flat plate. The two indicates that there are two
  sections (root and tip) for which this function is defined
28 #(if xle, yle, & zle had three points, you would increase this
  second digit to be three).
29
30 ns = n # number of spanwise panels. Each panel is a trapezoidal
  shape bounded by the chords at the inner and outer
31 #edges, as well as the leading and trailing edges of the wing.
  Each panel has a different shape.
32 nc = 1 # number of chordwise panels. Similar to spanwise panels,
  but in the chordwise direction.
33 spacing_s = Sine() # spanwise discretization scheme. This scheme
  uses a sine function to distribute the panels along
```

```

34 #the span of the wing, resulting in a higher concentration of
    panels near the wing root and tip, where aerodynamic
35 #effects are the greatest. In other words, the panels closest to
    the root and tips are narrower than those in between
36 #them (in the spanwise direction).
37 spacing_c = Uniform() # chordwise discretization scheme. Instead of
    concentrating the panels near the edges (See
38 #Sine function above), the panels are uniform throughout the wing
    (in the direction of the chord line).
39
40 grid, ratio = wing_to_grid(xle, yle, zle, chord, theta, phi, ns,
    nc;
41 fc = fc, spacing_s=spacing_s, spacing_c=spacing_c, mirror=true)#
    This combines all the factors included in the ()
42 #in order to create a meshed, 3-D shape of the wing. It stores
    this shape in the variable "grid", which is used later
43 #in the code to perform aerodynamic analysis. The "ratio" variable
    is a vector storing chordwise panel stretching ratios
44 #(the shape of the panels in the chordwise direction). This is
    used to compute the aerodynamic influence matrices
45 #and the scale vortex strengths during the analysis. The mirror
    option is set to true, meaning that the wing is mirrored
46 #across the aircraft's centerline to create a full wing (instead
    of just a half-wing).
47
48 grids = [grid] #list of grids in the system (only one wing in this
    case). It simply stores the grid created above
49 #in a vector(useful for organizing and managing multiple grids in
    a more complex analysis involving several lifting
50 #surfaces)
51 ratios = [ratio] #list of ratios in the system (only one wing in
    this case). It simply stores the grid created above
52 #in a vector(useful for organizing and managing multiple grids in
    a more complex analysis involving several lifting
53 #surfaces)
54 system = System(grids; ratios) # Combines grids and ratios in a
    System object, which is a data structure used to store
55 #and manage all the lifting surfaces in the analysis. This object
    is essential for performing aerodynamic calculations.
56
57 Sref = 0.785 # reference area
58 cref = 1 # reference chord
59 rref = [0.50, 0.0, 0.0] # reference location for rotations/moments
    (typically the center of gravity)
60 Vinf = 100.0 # reference velocity (magnitude)

```

```

61 ref = Reference(Sref, cref, bref, rref, Vinf) #Creates a Reference
    object that stores the reference parameters used
62 #in the aerodynamic analysis.
63
64 alpha = 1.0*pi/180 # angle of attack
65 beta = 0.0 # sideslip angle. The angle between the aircrafts
    forward pointing axis and the actual direction the air
66 #is coming from. When you set beta = 0.0, you are telling the
    solver the aircraft has no sideways motion relative to
67 #the airflow.
68 Omega = [0.0, 0.0, 0.0] # rotational velocity around the reference
    location (rref). This vector represents the angular
69 #velocity of the aircraft in three-dimensional space, with
    components along the x, y, and z axes. Setting it to
70 #[0.0, 0.0, 0.0] indicates that the aircraft is not rotating.
71 fs = Freestream(Vinf, alpha, beta, Omega) #Combines the freestream
    parameters into a Freestream object, which is used
72 #to define the incoming airflow conditions for the aerodynamic
    analysis.
73
74 symmetric = false #When set to false, the solver considers both
    sides of the aircraft separately, allowing for asymmetric
75 #flow conditions and forces. This is important for accurately
    capturing the aerodynamic behavior of the aircraft,
76 #especially during maneuvers (anytime theres a rolling and yawing
    moment) or in the presence of crosswinds. When set
77 #to false, the solver does not assume symmetry about the aircraft's
    centerline and cancel out forces and moments.
78
79 steady_analysis!(system, ref, fs; symmetric)
80 #This function performs a steady-state aerodynamic analysis on the
    lifting surfaces defined in the "system" object,
81 #using the reference parameters in the "ref" object and the
    freestream conditions in the "fs" object (Simply put,
82 #it takes all the information found above, and performed an
    aerodynamic analysis). The analysis assumes that the flow
83 #conditions are constant over time (steady) and that the aircraft
    is not symmetric about its centerline (symmetric=false).
84 #The results of the analysis, including aerodynamic forces and
    moments, are stored within the "system" object for further
85 #evaluation
86 CF, CM = body_forces(system; frame=Wind()) #This creates the force
    and moment vectors in the wind frame of reference.
87 #Of the force vector, the x-component is the drag force, the y-
    component is the side force, and the z-component is

```

```

88 #the lift force. Of the moment vector, the x-component is the
    rolling moment, the y-component is the pitching moment,
89 #and the z-component is the yawing moment.
90
91 # extract aerodynamic forces from the vectors created above.
92 CD, CY, CL = CF
93 Cl, Cm, Cn = CM
94 CDiff = far_field_drag(system)#Computes the drag contribution from
    the far-field induced drag, which is the drag caused by the
95 #vortices shed into the wake behind the lifting surfaces. This
    function calculates the far-field drag based on the vortex
    strengths
96 #and wake geometry determined during the steady-state analysis.
    The result is stored in the variable "CDiff".
97 #Simply put --> It calculated the coefficient of induced drag.
98
99 # calculate lifting line coefficients
100 cf, cm = lifting_line_coefficients(system; frame=Body()) #cf is
    the calculated section force coefficients along the
101 #span, and cm is the calculated section moment coefficients along
    the span. They are distributions and not total forces.
102 #frame=body means that its using the reference frame of the
    aircraft body (as opposed to wind frame used earlier).
103 cf, cm = lifting_line_coefficients(system; frame=Wind())
104 Gamma = system.          #This function retrieves the vortex strengths
    for the lifting surfaces defined in the
105 filtered_Gamma = []
106 density = 0.38
107 total_lift_elyptical = 0
108 qp = 0.5 * density * Vinf^2
109 Dpsu_distribution = []
110 for i in 1:length(cf)
111     Dpsu = q * chord[i] * cf[i][1]    # Force per meter (N/m)
112     push!(Dpsu_distribution, Dpsu)
113 end
114 for i in 1:length(Gamma)
115     if i % 1 == 0
116         Gammepsu = Gamma[i]
117         # Lpsu1 = 0.5 * Gammepsu * density * Vinf^2 * Sref
118         Lpsu1 = Gammepsu * density * Vinf
119         global total_lift_elyptical += Lpsu1
120         push!(filtered_Gamma, Lpsu1)
121         # Dpsu = qp .* chord[i] * cf[i] # Force per meter (N/m)
122         # push!(Dpsu_distribution, Dpsu)
123     end

```



```

124 end
125 # calculate lifting line coefficients
126 cf, cm = lifting_line_coefficients(system; frame=Body()) #cf is
    the calculated section force coefficients along the
127 #span, and cm is the calculated section moment coefficients along
    the span. They are distributions and not total forces.
128 #frame=body means that its using the reference frame of the
    aircraft body (as opposed to wind frame used earlier).
129
130 reverse!(filtered_Gamma)
131 xax = [39.3701 * grid[2,1,i+1] for i in 1:length(filtered_Gamma)]
132 plot!(xax, filtered_Gamma, label="Elliptical_Wing", linewidth=1)
133 #println("Total Lift for Tapered Chord Wing: ",
    total_lift_elyptical)
134 # println("Total Lift for an Elliptical Wing: ", trapz(xax,
    filtered_Gamma))
135
136 # L0 = maximum(filtered_Gamma)
137 # b = bref
138 # ideal_elyptical = []
139 # for i in 2:length(grid[2,1,:])
140 #     liftpsu = L0 * sqrt(1 - ((2 * grid[2,1,i])/(b))^2)
141 #     push!(ideal_elyptical, liftpsu)
142 # end
143
144
145 #plot!(xax, ideal_elyptical, label="Ideal Elyptical Wing",
    linewidth=3)
146 # pe1 = plot(xax, ideal_elyptical, ylims=(0, .55), label="Ideal
    Elliptical", linewidth=2, xlabel="Spanwise Location (m)",
    ylabel="Lift per Unit Span (N/m)")
147 # plot!(xax, filtered_Gamma, label="Actual Elliptical", linewidth
    =2)
148 # plot(pcc, pe1, layout=(2,1))
149 plot(xax, filtered_Gamma, ylims=(0, 2.5), label="Ideal_Elliptical"
    , linewidth=2, xlabel="Spanwise_Location(in)", ylabel="Lift(N
    )")
150 # println("Efficiency - Elyptical: ", CF[3]/CF[1]) #Lift to Drag
    Ratio
151 # AR = (bref^2)/Sref
152 # println((CF[3]^2)/(pi * CDiff * AR))
153 #println(CF[3])
154 #teapz(b_span, CL.l) (Find the integral of the lift distribution
    to get total lift)
155

```

```

156
157
158
159
160
161
162
163
164
165
166 # straight section of the beam
167 L_b1 = b_half/2 * 39.3701 # length of straight section of the
    beam in inches
168 r_b1 = [0, 0, 0] # starting point of straight section of the beam
169 nelem_b1 = n      2 # number of elements in the straight section of
    the beam
170 lengths_b1, xp_b1, xm_b1, Cab_b1 = discretize_beam(L_b1, r_b1,
    nelem_b1)
171
172 # swept section of the beam
173 L_b2 = b_half/2 * 39.3701 # length of swept section of the beam
174 r_b2 = [L_b1, 0, 0] # starting point of swept section of the beam
175 nelem_b2 = n      2 # number of elements in swept section of the
    beam
176 e1 = [1, 0, 0] # axis 1
177 e2 = [0, 1, 0] # axis 2
178 e3 = [0, 0, 1] # axis 3
179 frame_b2 = hcat(e1, e2, e3) # transformation matrix from local to
    body frame
180 lengths_b2, xp_b2, xm_b2, Cab_b2 = discretize_beam(L_b2, r_b2,
    nelem_b2;
181     frame = frame_b2)
182
183
184 # combine elements and points into one array
185 nelem = nelem_b1 + nelem_b2 # total number of elements
186 points = vcat(xp_b1, xp_b2[2:end]) # all points in our assembly
187 start = 1:nelem_b1 + nelem_b2 # starting point of each beam
    element in our assembly
188 stop = 2:nelem_b1 + nelem_b2 + 1 # ending point of each beam
    element in our assembly
189 lengths = vcat(lengths_b1, lengths_b2) # length of each beam
    element in our assembly
190 midpoints = vcat(xm_b1, xm_b2) # midpoint of each beam element in
    our assembly

```

```

191 Cab = vcat(Cab_b1, Cab_b2) # transformation matrix from local to
    body frame
192                                     # for each beam element in our assembly
193
194
195
196 # elem_lengths = lengths # already in inches
197 elem_mid = [midpoints[i][1] for i in 1:nelem]
198 q_lift = [(filtered_Gamma[i] / lengths[i]) * 0.2242 for i in 1:
    nelem]
199
200 # cf, cm = lifting_line_coefficients(system; frame=Wind())
201 # wing_cf = cf[1]
202 # qp = 0.5 * density * Vinf^2
203 # dpsu_distribution = []
204 # for i in 1:length(elem_mid)
205 #     local_chord = chord[i]
206 #     cd_local = wing_cf[i][1]
207 #     Dpsu = qp * local_chord * cd_local
208 #     push!(dpsu_distribution, Dpsu)
209 # end
210 # drag_plot = plot(elem_mid, dpsu_distribution, label="Drag
    Distribution", xlabel="Distance along the beam (in)", ylabel="
    Drag per Unit Length (N/in)", title="Drag Distribution Along
    the Beam")
211 # display(drag_plot)
212
213
214
215 # cross section
216 h = 1 # inch
217 w = 1.9685 # inch
218
219 # material properties
220 E = 29000000 # lb/in^2
221     = 0.28
222     = 0.284 # lb sec^2/in^4
223
224 # shear and torsion correction factors
225 ky = 0.83
226 kz = 0.83
227 kt = 1
228
229 A = h*w
230 Iyy = w*h^3/12

```

```

231 Izz = w^3*h/12
232 J = Iyy + Izz
233 # apply corrections
234 Ay = A/ky
235 Az = A/kz
236 Jx = J/kt
237
238 G = E/(2*(1+ ))
239 # println()
240 # println(q_lift)
241 # println()
242 compliance = fill(Diagonal([1/(E*A), 1/(G*Ay), 1/(G*Az), 1/(G*Jx),
243     1/(E*Iyy),
244     1/(E*Izz)]), nelelem)#creates a 6x6 compliance matrix for each
245     element in the assembly, where the diagonal
246     #entries correspond to the inverse of the stiffness of the
247     element in each degree of freedom. The
248     #compliance matrix is used to relate forces and moments to
249     displacements and rotations in the beam
250     #elements.
251
252 # println(compliance[1])
253 for ww in 1:n
254     local w = max(chord[ww], 0.01) * 39.3701 # avoid zero chord
255     local A = h*w
256     local Iyy = w*h^3/12
257     local Izz = w^3*h/12
258     local J = Iyy + Izz
259     local Ay = A/ky
260     local Az = A/kz
261     local Jx = J/kt
262     compliance[ww] = Diagonal([1/(E*A), 1/(G*Ay), 1/(G*Az), 1/(G*
263         Jx), 1/(E*Iyy), 1/(E*Izz)])
264 end
265
266 mass = fill(Diagonal([ *A, *A, *A, *J, *Iyy, *Izz]),
267     nelelem)#creates a 6x6 mass matrix for each element
268 #in the assembly, where the diagonal entries correspond to the
269 mass of the element in each degree of freedom.
270 #The mass matrix is used to relate accelerations to forces and
271 moments in the beam elements.
272 total_mass_elyptical = 0
273 for ww in 1:n
274     local w = max(chord[ww], 0.01) * 39.3701 # avoid zero chord
275     local A = h*w

```

```

268     local Iyy = w*h^3/12
269     local Izz = w^3*h/12
270     local J = Iyy + Izz
271     local Ay = A/ky
272     local Az = A/kz
273     local Jx = J/kt
274     mass[ww] = Diagonal([ *A, *A, *A, *J, *Iyy, *Izz])
275     global total_mass_elyptical += *A*lengths[ww] # mass per
        unit length * length of element
276 end
277 println("Total_Mass_of_the_Elyptical_Wing:", total_mass_elyptical
        , "lb")
278
279 assembly = Assembly(points, start, stop;
280     compliance = compliance,
281     mass = mass,
282     frames = Cab,
283     lengths = lengths,
284     midpoints = midpoints)
285
286
287 # mkpath("rotating-geometry")
288 # write_vtk("rotating-geometry/rotating-geometry", assembly)
289
290 # ADDING POINT MASSES
291 # m = 10 # mass
292 # p = zeros(3) # relative location
293 # J = zeros(3,3) # inertia matrix (about the point mass center of
        gravity)
294
295 # # create dictionary of point masses
296 # point_masses = Dict(
297 #     nelem+1 => PointMass(m, p, J)
298 # )
299
300 # DISTRIBUTED LOADS FROM AERODYNAMIC LIFT (using right-end value)
301 distributed_loads = Dict{Int64, DistributedLoads{Float64}}{ }()
302 for ielem in 1:nelem
303     local q = q_lift[ielem] # lb/in, force per unit length for
        this element
304     distributed_loads[ielem] = DistributedLoads(assembly, ielem;
305         fz = (s) -> q # constant load over the element
306     )
307 end
308

```

```

309
310 # # DISTRIBUTED LOADS THAT ROTATE WITH THE BEAM
311 # distributed_loads = Dict{Int64, DistributedLoads{Float64}}{ }()
312 # for ielem in 1:nelem
313 #     distributed_loads[ielem] = DistributedLoads(assembly, ielem;
314 #         fz_follower = (s) -> 1)
315 # end
316
317 # Defining PRESCRIBED CONDITIONS
318 # create FIXED BOUNDARY CONDITION ON THE LEFT SIDE (ROOT) OF THE
    BEAM-->
319 prescribed_conditions = Dict(
320     # left support: fix all translations, block rotation about
        beam axis
321     1 => PrescribedConditions(ux = 0, uy = 0, uz = 0, theta_x = 0,
        theta_y = 0, theta_z = 0)
322 )
323
324 # PREALLOCATING MEMORY FOR OUR ANALYSIS(CAN USE StaticSystem,
    DynamicSystem, or ExpandedSystem(when using the mass matrix for
    differential equations))
325 system = DynamicSystem(assembly)
326
327
328
329
330
331
332
333
334 # Performing a Steady State ANALYSIS
335 rpm = [0.0]
336
337 # linear_states = Vector{AssemblyState{Float64}}{undef, length(rpm
    )}
338 # for i = 1:length(rpm)
339
340 #     # global frame rotation
341 #     w0 = [0, 0, rpm[i]*(2*pi)/60]
342
343 #     # perform linear steady state analysis
344 #     _, state, converged = steady_state_analysis!(system,
        assembly,
345 #         angular_velocity = w0,
346 #         prescribed_conditions = prescribed_conditions,

```

```

347 #         distributed_loads = distributed_loads,
348 #         linear = true)
349
350 #     # save result
351 #     linear_states[i] = state
352
353 # end
354
355 # reset_state!(system)
356
357 nonlinear_states = Vector{AssemblyState{Float64}}(undef, length(
    rpm))
358 for i = 1:length(rpm)
359
360     # global frame rotation
361     w0 = [0, 0, rpm[i]*(2*pi)/60]
362
363     # perform nonlinear steady state analysis
364     _, state, converged = steady_state_analysis!(system, assembly,
365         angular_velocity = w0,
366         distributed_loads = distributed_loads,
367         prescribed_conditions = prescribed_conditions)
368
369     # save result
370     nonlinear_states[i] = state
371
372 end
373 # ^^Linear solution assumes small deformations and rotations,
    while
374 #nonlinear solution accounts for large deformations and rotations.
375 #The linear solution may not be accurate at higher rotation speeds
376 #where the beam experiences significant deformation, while the
    nonlinear
377 #solution provides a more accurate representation of the beam's
    behavior
378 #under these conditions.
379 # ^^Linear assumes no geometric stiffening, no coupling between
    bending,
380 #torsion and axial stretch, and centrifugal forces are linearized
    .
381 #Nonlinear accounts for all of these effects, which become more
    significant
382 #at higher rotation speeds.
383 #Nonlinear will update stiffness as ther beam deflects
384

```

```

385 # i = 1 # first RPM case
386 # # vertical displacement u_z at every node
387 # uz_all = [linear_states[i].points[n].u[3] for n in 1:length(
    points)]
388 # # x coordinates of each node
389 # xvaluesnumerical = [points[n][1] for n in 1:length(points)]
390 # println(uz_all)
391
392
393 # #####LINEAR
394 # i = 1 # for example, the 1st RPM case
395 # uz_all = [linear_states[i].points[n].u[3] for n in 1:length(
    points)]
396 # uy_all = [linear_states[i].points[n].u[2] for n in 1:length(
    points)]
397 # ux_all = [linear_states[i].points[n].u[1] for n in 1:length(
    points)]
398 # xvaluesnumerical = [points[aa][1] for aa in 1:length(points)]
399 # thetaxGX = [linear_states[i].points[n].theta[2] for n in 1:
    length(points)]
400 # #print(length(linear_states[i].elements))
401 # #println(linear_states[i].elements[16].Mi)
402 # moments = [linear_states[i].elements[n].Mi[2] for n in 1:length(
    linear_states[i].elements)]
403 # xvalues12 = [midpoints[n][1] for n in 1:length(linear_states[i].
    elements)]
404 # shears = [linear_states[i].elements[n].Fi[3] for n in 1:length(
    linear_states[i].elements)]
405
406 # #####NONLINEAR
407 i = 1 # for example, the 1st RPM case
408 uz_all = [nonlinear_states[i].points[n].u[3] for n in 1:length(
    points)]
409 uy_all = [nonlinear_states[i].points[n].u[2] for n in 1:length(
    points)]
410 ux_all = [nonlinear_states[i].points[n].u[1] for n in 1:length(
    points)]
411 xvaluesnumerical = [points[aa][1] for aa in 1:length(points)]
412 thetaxGX = [nonlinear_states[i].points[n].theta[2] for n in 1:
    length(points)]
413 #print(length(linear_states[i].elements))
414 #println(linear_states[i].elements[16].Mi)
415 moments = [nonlinear_states[i].elements[n].Mi[2] for n in 1:length
    (nonlinear_states[i].elements)]

```



```

416 xvalues12 = [midpoints[n][1] for n in 1:length(nonlinear_states[i
    ].elements)]
417 shears = [nonlinear_states[i].elements[n].Fi[3] for n in 1:length(
    nonlinear_states[i].elements)]
418 println("Total_Drag_Coeffiecient_for_Elyptical_Wing:", CDiff)
419 println()
420
421
422
423
424
425
426
427
428
429
430 #
    #####

431
432
433
434
435
436
437
438
439
440
441
442 ###Rectangular Wing
443 bref = 10 # reference span
444 n = 20      # number of spanwise panels
445 xle = zeros(n)      # no sweep
446     = range(0,      /2; length=n)
447 b_half = bref*0.5
448 yle = range(0, b_half; length=n)
449 zle = zeros(n)
450 chord = ones(n)
451 theta = zeros(n)
452 phi   = zeros(n)
453 fc = fill((xc) -> 0.0, n)
454 # camberline function for each section (y/c = f(x/c))
455 #This function describes the camberline of the airfoil sections
    along the span of the wing, creating the upper and

```

```

456 #lower edges of the airfoils. This function is set to zero (the
    first number), indicating that the wing has no camber
457 #and is a flat plate. The two indicates that there are two
    sections (root and tip) for which this function is defined
458 #(if xle, yle, & zle had three points, you would increase this
    second digit to be three).
459
460 ns = n # number of spanwise panels. Each panel is a trapezoidal
    shape bounded by the chords at the inner and outer
461 #edges, as well as the leading and trailing edges of the wing.
    Each panel has a different shape.
462 nc = 1 # number of chordwise panels. Similar to spanwise panels,
    but in the chordwise direction.
463 spacing_s = Sine() # spanwise discretization scheme. This scheme
    uses a sine function to distribute the panels along
464 #the span of the wing, resulting in a higher concentration of
    panels near the wing root and tip, where aerodynamic
465 #effects are the greatest. In other words, the panels closest to
    the root and tips are narrower than those in between
466 #them (in the spanwise direction).
467 spacing_c = Uniform() # chordwise discretization scheme. Instead of
    concentrating the panels near the edges (See
468 #Sine function above), the panels are uniform throughout the wing
    (in the direction of the chord line).
469
470 grid, ratio = wing_to_grid(xle, yle, zle, chord, theta, phi, ns,
    nc;
471 fc = fc, spacing_s=spacing_s, spacing_c=spacing_c, mirror=true)#
    This combines all the factors included in the ()
472 #in order to create a meshed, 3-D shape of the wing. It stores
    this shape in the variable "grid", which is used later
473 #in the code to perform aerodynamic analysis. The "ratio" variable
    is a vector storing chordwise panel stretching ratios
474 #(the shape of the panels in the chordwise direction). This is
    used to compute the aerodynamic influence matrices
475 #and the scale vortex strengths during the analysis. The mirror
    option is set to true, meaning that the wing is mirrored
476 #across the aircraft's centerline to create a full wing (instead
    of just a half-wing).
477
478 grids = [grid] #list of grids in the system (only one wing in this
    case). It simply stores the grid created above
479 #in a vector(useful for organizing and managing multiple grids in
    a more complex analysis involving several lifting
480 #surfaces)

```

```

481 ratios = [ratio] #list of ratios in the system (only one wing in
    this case). It simply stores the grid created above
482 #in a vector(useful for organizing and managing multiple grids in
    a more complex analysis involving several lifting
483 #surfaces)
484 system = System(grids; ratios) # Combines grids and ratios in a
    System object, which is a data structure used to store
485 #and manage all the lifting surfaces in the analysis. This object
    is essential for performing aerodynamic calculations.
486
487 Sref = 0.785 # reference area
488 cref = 1 # reference chord\
489 rref = [0.50, 0.0, 0.0] # reference location for rotations/moments
    (typically the center of gravity)
490 # Vinf = 10.0 # reference velocity (magnitude)
491 ref = Reference(Sref, cref, bref, rref, Vinf) #Creates a Reference
    object that stores the reference parameters used
492 #in the aerodynamic analysis.
493
494 alpha = 1.0*pi/180 # angle of attack
495 beta = 0.0 # sideslip angle. The angle between the aircrafts
    forward pointing axis and the actual direction the air
496 #is coming from. When you set beta = 0.0, you are telling the
    solver the aircraft has no sideways motion relative to
497 #the airflow.
498 Omega = [0.0, 0.0, 0.0] # rotational velocity around the reference
    location (rref). This vector represents the angular
499 #velocity of the aircraft in three-dimensional space, with
    components along the x, y, and z axes. Setting it to
500 #[0.0, 0.0, 0.0] indicates that the aircraft is not rotating.
501 fs = Freestream(Vinf, alpha, beta, Omega) #Combines the freestream
    parameters into a Freestream object, which is used
502 #to define the incoming airflow conditions for the aerodynamic
    analysis.
503
504 symmetric = false #When set to false, the solver considers both
    sides of the aircraft separately, allowing for asymmetric
505 #flow conditions and forces. This is important for accurately
    capturing the aerodynamic behavior of the aircraft,
506 #especially during maneuvers (anytime theres a rolling and yawing
    moment) or in the presence of crosswinds. When set
507 #to false, the solver does not assume symmetry about the aircraft's
    centerline and cancel out forces and moments.
508
509 steady_analysis!(system, ref, fs; symmetric)

```

```

510 #This function performs a steady-state aerodynamic analysis on the
    lifting surfaces defined in the "system" object,
511 #using the reference parameters in the "ref" object and the
    freestream conditions in the "fs" object (Simply put,
512 #it takes all the information found above, and performed an
    aerodynamic analysis). The analysis assumes that the flow
513 #conditions are constant over time (steady) and that the aircraft
    is not symmetric about its centerline (symmetric=false).
514 #The results of the analysis, including aerodynamic forces and
    moments, are stored within the "system" object for further
515 #evaluation
516 CF, CM = body_forces(system; frame=Wind()) #This creates the force
    and moment vectors in the wind frame of reference.
517 #Of the force vector, the x-component is the drag force, the y-
    component is the side force, and the z-component is
518 #the lift force. Of the moment vector, the x-component is the
    rolling moment, the y-component is the pitching moment,
519 #and the z-component is the yawing moment.
520
521 # extract aerodynamic forces from the vectors created above.
522 CD, CY, CL = CF
523 Cl, Cm, Cn = CM
524
525 CDiff = far_field_drag(system)#Computes the drag contribution from
    the far-field induced drag, which is the drag caused by the
526 #vortices shed into the wake behind the lifting surfaces. This
    function calculates the far-field drag based on the vortex
    strengths
527 #and wake geometry determined during the steady-state analysis.
    The result is stored in the variable "CDiff".
528 #Simply put --> It calculated the coefficient of induced drag.
529
530 # calculate lifting line coefficients
531 cf, cm = lifting_line_coefficients(system; frame=Body()) #cf is
    the calculated section force coefficients along the
532 #span, and cm is the calculated section moment coefficients along
    the span. They are distributions and not total forces.
533 #frame=body means that its using the reference frame of the
    aircraft body (as opposed to wind frame used earlier).
534
535 Gamma = system. #This function retrieves the vortex strengths
    for the lifting surfaces defined in the
536 filtered_Gamma = []
537 density = 0.38
538 total_lift_elyptical = 0

```

```

539 for i in 1:length(Gamma)
540     if i % 1 == 0
541         Gammepsu = Gamma[i]
542         # Lpsu1 = 0.5 * Gammepsu * density * Vinf^2 * Sref
543         Lpsu1 = Gammepsu * density * Vinf
544         global total_lift_elyptical += Lpsu1
545         push!(filtered_Gamma, Lpsu1)
546     end
547 end
548 reverse!(filtered_Gamma)
549 xax = [39.3701 * grid[2,1,i+1] for i in 1:length(filtered_Gamma)]
550 plot!(xax, filtered_Gamma, label="Elliptical_Wing", linewidth=1)
551 #println("Total Lift for Tapered Chord Wing: ",
552         total_lift_elyptical)
553 # println("Total Lift for an Elliptical Wing: ", trapz(xax,
554         filtered_Gamma))
555
556 # L0 = maximum(filtered_Gamma)
557 # b = bref
558 # ideal_elyptical = []
559 # for i in 2:length(grid[2,1,:])
560 #     liftpsu = L0 * sqrt(1 - ((2 * grid[2,1,i])/(b))^2)
561 #     push!(ideal_elyptical, liftpsu)
562 # end
563
564 #plot!(xax, ideal_elyptical, label="Ideal Elyptical Wing",
565         linewidth=3)
566 # pe1 = plot(xax, ideal_elyptical, ylims=(0, .55), label="Ideal
567         Elliptical", linewidth=2, xlabel="Spanwise Location (m)",
568         ylabel="Lift per Unit Span (N/m)")
569 # plot!(xax, filtered_Gamma, label="Actual Elliptical", linewidth
570         =2)
571 # plot(pcc, pe1, layout=(2,1))
572 plot(xax, filtered_Gamma, ylims=(0, 2.5), label="Ideal_Elliptical"
573     , linewidth=2, xlabel="Spanwise_Location(in)", ylabel="Lift(N
574     )")
575 # println("Efficiency - Elyptical: ", CF[3]/CF[1]) #Lift to Drag
576         Ratio
577 # AR = (bref^2)/Sref
578 # println((CF[3]^2)/(pi * CDiff * AR))
579 #println(CF[3])
580 #teapz(b_span, CL.l) (Find the integral of the lift distribution
581         to get total lift)
582
583

```

```

574
575
576
577
578
579
580
581
582
583
584 # straight section of the beam
585 L_b1 = b_half/2 * 39.3701 # length of straight section of the
    beam in inches
586 r_b1 = [0, 0, 0] # starting point of straight section of the beam
587 nelem_b1 = n      2 # number of elements in the straight section of
    the beam
588 lengths_b1, xp_b1, xm_b1, Cab_b1 = discretize_beam(L_b1, r_b1,
    nelem_b1)
589
590 # swept section of the beam
591 L_b2 = b_half/2 * 39.3701 # length of swept section of the beam
592 r_b2 = [L_b1, 0, 0] # starting point of swept section of the beam
593 nelem_b2 = n      2 # number of elements in swept section of the
    beam
594 e1 = [1, 0, 0] # axis 1
595 e2 = [0, 1, 0] # axis 2
596 e3 = [0, 0, 1] # axis 3
597 frame_b2 = hcat(e1, e2, e3) # transformation matrix from local to
    body frame
598 lengths_b2, xp_b2, xm_b2, Cab_b2 = discretize_beam(L_b2, r_b2,
    nelem_b2;
599     frame = frame_b2)
600
601
602 # combine elements and points into one array
603 nelem = nelem_b1 + nelem_b2 # total number of elements
604 points = vcat(xp_b1, xp_b2[2:end]) # all points in our assembly
605 start = 1:nelem_b1 + nelem_b2 # starting point of each beam
    element in our assembly
606 stop = 2:nelem_b1 + nelem_b2 + 1 # ending point of each beam
    element in our assembly
607 lengths = vcat(lengths_b1, lengths_b2) # length of each beam
    element in our assembly
608 midpoints = vcat(xm_b1, xm_b2) # midpoint of each beam element in
    our assembly

```

```

609 Cab = vcat(Cab_b1, Cab_b2) # transformation matrix from local to
    body frame
610                                     # for each beam element in our assembly
611
612
613
614 # elem_lengths = lengths # already in inches
615 elem_midR = [midpoints[i][1] for i in 1:nelem]
616 q_liftR = [(filtered_Gamma[i] / lengths[i]) * 0.2242 for i in 1:
    nelem]
617
618
619
620 # cross section
621 h = 1 # inch
622 w = 1 * 39.3701 # inch
623
624 # material properties
625 E = 29000000 # lb/in^2
626     = 0.28
627     = 0.284 # lb sec^2/in^4
628
629 # shear and torsion correction factors
630 ky = 0.83
631 kz = 0.83
632 kt = 1
633
634 A = h*w
635 Iyy = w*h^3/12
636 Izz = w^3*h/12
637 J = Iyy + Izz
638 # apply corrections
639 Ay = A/ky
640 Az = A/kz
641 Jx = J/kt
642
643 G = E/(2*(1+ ))
644
645 compliance = fill(Diagonal([1/(E*A), 1/(G*Ay), 1/(G*Az), 1/(G*Jx),
    1/(E*Iyy),
646     1/(E*Izz)]), nelem)#creates a 6x6 compliance matrix for each
    element in the assembly, where the diagonal
647     #entries correspond to the inverse of the stiffness of the
    element in each degree of freedom. The

```

```

648     #compliance matrix is used to relate forces and moments to
        displacements and rotations in the beam
649     #elements.
650
651 mass = fill(Diagonal([ *A,    *A,    *A,    *J,    *Iyy,    *Izz]),
        nelelem)#creates a 6x6 mass matrix for each element
652 #in the assembly, where the diagonal entries correspond to the
        mass of the element in each degree of freedom.
653 #The mass matrix is used to relate accelerations to forces and
        moments in the beam elements.
654 total_mass_rectangular = chord[1] * 39.3701 * h *      * sum(lengths
        ) # mass per unit length * total length of the beam
655 println("Total_Mass_of_the_Rectangular_Wing:",
        total_mass_rectangular, "lb")
656
657 assembly = Assembly(points, start, stop;
658     compliance = compliance,
659     mass = mass,
660     frames = Cab,
661     lengths = lengths,
662     midpoints = midpoints)
663
664
665 # mkpath("rotating-geometry")
666 # write_vtk("rotating-geometry/rotating-geometry", assembly)
667
668 # ADDING POINT MASSES
669 # m = 10 # mass
670 # p = zeros(3) # relative location
671 # J = zeros(3,3) # inertia matrix (about the point mass center of
        gravity)
672
673 # # create dictionary of point masses
674 # point_masses = Dict(
675 #     nelelem+1 => PointMass(m, p, J)
676 # )
677
678 # DISTRIBUTED LOADS FROM AERODYNAMIC LIFT (using right-end value)
679 distributed_loads = Dict{Int64, DistributedLoads{Float64}}{ }
680 for ielem in 1:nelelem
681     local q = q_liftR[ielem]    # lb/in, force per unit length for
        this element
682     distributed_loads[ielem] = DistributedLoads(assembly, ielem;
683         fz = (s) -> q    # constant load over the element
684     )

```



```

685 end
686
687
688 # # DISTRIBUTED LOADS THAT ROTATE WITH THE BEAM
689 # distributed_loads = Dict{Int64, DistributedLoads{Float64}}{ }()
690 # for ielem in 1:nelem
691 #     distributed_loads[ielem] = DistributedLoads(assembly, ielem;
692 #         fz_follower = (s) -> 1)
693 # end
694
695 # Defining PRESCRIBED CONDITIONS
696 # create FIXED BOUNDARY CONDITION ON THE LEFT SIDE (ROOT) OF THE
    BEAM-->
697 prescribed_conditions = Dict(
698     # left support: fix all translations, block rotation about
        beam axis
699     1 => PrescribedConditions(ux = 0, uy = 0, uz = 0, theta_x = 0,
        theta_y = 0, theta_z = 0)
700 )
701
702 # PREALLOCATING MEMORY FOR OUR ANALYSIS(CAN USE StaticSystem,
        DynamicSystem, or ExpandedSystem(when using the mass matrix for
        differential equations))
703 system = DynamicSystem(assembly)
704
705
706
707
708
709
710
711
712 # Performing a Steady State ANALYSIS
713 rpm = [0.0]
714
715 # linear_states = Vector{AssemblyState{Float64}}{ }(undef, length(rpm)
        )
716 # for i = 1:length(rpm)
717
718 #     # global frame rotation
719 #     w0 = [0, 0, rpm[i]*(2*pi)/60]
720
721 #     # perform linear steady state analysis
722 #     _, state, converged = steady_state_analysis!(system,
        assembly,

```

```

723 #         angular_velocity = w0,
724 #         prescribed_conditions = prescribed_conditions,
725 #         distributed_loads = distributed_loads,
726 #         linear = true)
727
728 #     # save result
729 #     linear_states[i] = state
730
731 # end
732
733 # reset_state!(system)
734
735 nonlinear_states = Vector{AssemblyState{Float64}}(undef, length(
    rpm))
736 for i = 1:length(rpm)
737
738     # global frame rotation
739     w0 = [0, 0, rpm[i]*(2*pi)/60]
740
741     # perform nonlinear steady state analysis
742     _, state, converged = steady_state_analysis!(system, assembly,
743         angular_velocity = w0,
744         distributed_loads = distributed_loads,
745         prescribed_conditions = prescribed_conditions)
746
747     # save result
748     nonlinear_states[i] = state
749
750 end
751
752
753 # i = 1 # first RPM case
754 # # vertical displacement u_z at every node
755 # uz_all = [linear_states[i].points[n].u[3] for n in 1:length(
    points)]
756 # # x coordinates of each node
757 # xvaluesnumerical = [points[n][1] for n in 1:length(points)]
758 # println(uz_all)
759
760
761 # #####LINEAR
762 # i = 1 # for example, the 1st RPM case
763 # uz_all = [linear_states[i].points[n].u[3] for n in 1:length(
    points)]

```

```

764 # uy_all = [linear_states[i].points[n].u[2] for n in 1:length(
      points)]
765 # ux_all = [linear_states[i].points[n].u[1] for n in 1:length(
      points)]
766 # xvaluesnumerical = [points[aa][1] for aa in 1:length(points)]
767 # thetaxGX = [linear_states[i].points[n].theta[2] for n in 1:
      length(points)]
768 # #print(length(linear_states[i].elements))
769 # #println(linear_states[i].elements[16].Mi)
770 # moments = [linear_states[i].elements[n].Mi[2] for n in 1:length(
      linear_states[i].elements)]
771 # xvalues12 = [midpoints[n][1] for n in 1:length(linear_states[i].
      elements)]
772 # shears = [linear_states[i].elements[n].Fi[3] for n in 1:length(
      linear_states[i].elements)]
773
774 #####NONLINEAR
775 i = 1 # for example, the 1st RPM case
776 uz_allR = [nonlinear_states[i].points[n].u[3] for n in 1:length(
      points)]
777 uy_allR = [nonlinear_states[i].points[n].u[2] for n in 1:length(
      points)]
778 ux_allR = [nonlinear_states[i].points[n].u[1] for n in 1:length(
      points)]
779 xvaluesnumericalR = [points[aa][1] for aa in 1:length(points)]
780 thetaxGXR = [nonlinear_states[i].points[n].theta[2] for n in 1:
      length(points)]
781 #print(length(linear_states[i].elements))
782 #println(linear_states[i].elements[16].Mi)
783 momentsR = [nonlinear_states[i].elements[n].Mi[2] for n in 1:
      length(nonlinear_states[i].elements)]
784 xvalues12R = [midpoints[n][1] for n in 1:length(nonlinear_states[i]
      ].elements)]
785 shearsR = [nonlinear_states[i].elements[n].Fi[3] for n in 1:length
      (nonlinear_states[i].elements)]
786
787
788 # wx = []
789 # thetax = []
790 # xvalues = []
791 # Mx = []
792 # Vx = []
793 # for i in 1:length(points)
794 #     q0 = 1
795 #     #E = 1.06e7 # lb/in^2

```

```

796 #     x = points[i][1]
797 #     L = b_half/2 * 39.3701
798 #     #println(x)
799 #     wxpart = ((q0 * L^4) / (24 * E * Iyy)) * ((x/L) - 2*(x^3/L
800 #         ^3) + (x^4/L^4))
801 #     thetaxpart = -(q0*L^3) / (24 * E * Iyy) * (1 - 6*(x^2/L^2) +
802 #         4*(x^3/L^3))
803 #     Mxpart = ((q0*L^2) / 2) * ((x/L) - (x^2/L^2))
804 #     Vxpart = ((q0*L) / 2) * (1 - 2*(x/L))
805 #     push!(wx, wxpart)
806 #     push!(xvalues, x)
807 #     push!(thetax, thetaxpart)
808 #     push!(Mx, Mxpart)
809 #     push!(Vx, Vxpart)
810 # end
811
812 # print(nonlinear_states[i].elements[1].Mi[2])
813 println("Total_Drag_Coeffiecient_for_Rectangular_Wing:", CDiff)
814 println()
815
816
817
818
819
820
821
822
823
824
825
826 #
827
828
829
830
831
832
833
834
835
836

```

```

837
838 ###Tapered Wing
839 bref = 10 # reference span
840 n = 20      # number of spanwise panels
841 xle = zeros(n)      # no sweep
842     = range(0,      /2; length=n)
843 b_half = bref*0.5
844 yle = range(0, b_half; length=n)
845 zle = zeros(n)
846 c_root = 1
847 chord = c_root .- (c_root .- 0.2) .* (yle ./ b_half) # linear
      taper from root to tip, where the tip chord is 0.5 times the
      root chord
848 theta = zeros(n)
849 phi    = zeros(n)
850 fc = fill((xc) -> 0.0, n)
851 # camberline function for each section (y/c = f(x/c))
852 #This function describes the camberline of the airfoil sections
      along the span of the wing, creating the upper and
853 #lower edges of the airfoils. This function is set to zero (the
      first number), indicating that the wing has no camber
854 #and is a flat plate. The two indicates that there are two
      sections (root and tip) for which this function is defined
855 #(if xle, yle, & zle had three points, you would increase this
      second digit to be three).
856
857 ns = n # number of spanwise panels. Each panel is a trapezoidal
      shape bounded by the chords at the inner and outer
858 #edges, as well as the leading and trailing edges of the wing.
      Each panel has a different shape.
859 nc = 1 # number of chordwise panels. Similar to spanwise panels,
      but in the chordwise direction.
860 spacing_s = Sine() # spanwise discretization scheme. This scheme
      uses a sine function to distribute the panels along
861 #the span of the wing, resulting in a higher concentration of
      panels near the wing root and tip, where aerodynamic
862 #effects are the greatest. In other words, the panels closest to
      the root and tips are narrower than those in between
863 #them (in the spanwise direction).
864 spacing_c = Uniform() # chordwise discretization scheme. Insted of
      concentrating the panels near the edges (See
865 #Sine function above), the panels are uniform throughout the wing
      (in the direction of the chord line).
866

```

```

867 grid, ratio = wing_to_grid(xle, yle, zle, chord, theta, phi, ns,
868   nc;
869 fc = fc, spacing_s=spacing_s, spacing_c=spacing_c, mirror=true)#
    This combines all the factors included in the ()
869 #in order to create a meshed, 3-D shape of the wing. It stores
    this shape in the variable "grid", which is used later
870 #in the code to perform aerodynamic analysis. The "ratio" variable
    is a vector storing chorwise panel stretching ratios
871 #(the shape of the panels in the chordwise direction). This is
    used to compute the aerodynamic influence matrices
872 #and the scale vortex strengths during the analysis. The mirror
    option is set to true, meaning that the wing is mirrored
873 #across the aircraft's centerline to create a full wing (instead
    of just a half-wing).
874
875 grids = [grid] #list of grids in the system (only one wing in this
    case). It simply stores the grid created above
876 #in a vector(useful for organizing and managing multiple grids in
    a more complex analysis involving several lifting
877 #surfaces)
878 ratios = [ratio] #list of ratios in the system (only one wing in
    this case). It simply stores the grid created above
879 #in a vector(useful for organizing and managing multiple grids in
    a more complex analysis involving several lifting
880 #surfaces)
881 system = System(grids; ratios) # Combines grids and ratios in a
    System object, which is a data structure used to store
882 #and manage all the lifting surfaces in the analysis. This object
    is essential for performing aerodynamic calculations.
883
884 Sref = 0.785 # reference area
885 cref = 1 # reference chord
886 rref = [0.50, 0.0, 0.0] # reference location for rotations/moments
    (typically the center of gravity)
887 Vinf = 100.0 # reference velocity (magnitude)
888 ref = Reference(Sref, cref, bref, rref, Vinf) #Creates a Reference
    object that stores the reference parameters used
889 #in the aerodynamic analysis.
890
891 alpha = 1.0*pi/180 # angle of attack
892 beta = 0.0 # sideslip angle. The angle between the aircrafts
    forward pointing axis and the actual direction the air
893 #is coming from. When you set beta = 0.0, you are telling the
    solver the aircraft has no sideways motion relative to
894 #the airflow.

```

```

895 Omega = [0.0, 0.0, 0.0] # rotational velocity around the reference
      location (rref). This vector represents the angular
896 #velocity of the aircraft in three-dimensional space, with
      components along the x, y, and z axes. Setting it to
897 #[0.0, 0.0, 0.0] indicates that the aircraft is not rotating.
898 fs = Freestream(Vinf, alpha, beta, Omega) #Combines the freestream
      parameters into a Freestream object, which is used
899 #to define the incoming airflow conditions for the aerodynamic
      analysis.
900
901 symmetric = false #When set to false, the solver considers both
      sides of the aircraft separately, allowing for asymmetric
902 #flow conditions and forces. This is important for accurately
      capturing the aerodynamic behavior of the aircraft,
903 #especially during maneuvers (anytime theres a rolling and yawing
      moment) or in the presence of crosswinds. When set
904 #to false, the solver does not assume symmetry about the aircraft's
      centerline and cancel out forces and moments.
905
906 steady_analysis!(system, ref, fs; symmetric)
907 #This function performs a steady-state aerodynamic analysis on the
      lifting surfaces defined in the "system" object,
908 #using the reference parameters in the "ref" object and the
      freestream conditions in the "fs" object (Simply put,
909 #it takes all the information found above, and performed an
      aerodynamic analysis). The analysis assumes that the flow
910 #conditions are constant over time (steady) and that the aircraft
      is not symmetric about its centerline (symmetric=false).
911 #The results of the analysis, including aerodynamic forces and
      moments, are stored within the "system" object for further
912 #evaluation
913 CF, CM = body_forces(system; frame=Wind()) #This creates the force
      and moment vectors in the wind frame of reference.
914 #Of the force vector, the x-component is the drag force, the y-
      component is the side force, and the z-component is
915 #the lift force. Of the moment vector, the x-component is the
      rolling moment, the y-component is the pitching moment,
916 #and the z-component is the yawing moment.
917
918 # extract aerodynamic forces from the vectors created above.
919 CD, CY, CL = CF
920 Cl, Cm, Cn = CM
921 CDiff = far_field_drag(system)#Computes the drag contribution from
      the far-field induced drag, which is the drag caused by the

```

```

922 #vortices shed into the wake behind the lifting surfaces. This
    function calculates the far-field drag based on the vortex
    strengths
923 #and wake geometry determined during the steady-state analysis.
    The result is stored in the variable "CDiff".
924 #Simply put --> It calculated the coefficient of induced drag.
925
926
927 # calculate lifting line coefficients
928 cf, cm = lifting_line_coefficients(system; frame=Body()) #cf is
    the calculated section force coefficients along the
929 #span, and cm is the calculated section moment coefficients along
    the span. They are distributions and not total forces.
930 #frame=body means that its using the reference frame of the
    aircraft body (as opposed to wind frame used earlier).
931
932 Gamma = system.          #This function retrieves the vortex strengths
    for the lifting surfaces defined in the
933 filtered_Gamma = []
934 density = 0.38
935 total_lift_elyptical = 0
936 for i in 1:length(Gamma)
937     if i % 1 == 0
938         Gammepsu = Gamma[i]
939         # Lpsu1 = 0.5 * Gammepsu * density * Vinf^2 * Sref
940         Lpsu1 = Gammepsu * density * Vinf
941         global total_lift_elyptical += Lpsu1
942         push!(filtered_Gamma, Lpsu1)
943     end
944 end
945 reverse!(filtered_Gamma)
946 xax = [39.3701 * grid[2,1,i+1] for i in 1:length(filtered_Gamma)]
947 # plot!(xax, filtered_Gamma, label="Elliptical Wing", linewidth=1)
948 #println("Total Lift for Tapered Chord Wing: ",
    total_lift_elyptical)
949 # println("Total Lift for an Elliptical Wing: ", trapz(xax,
    filtered_Gamma))
950
951 # L0 = maximum(filtered_Gamma)
952 # b = bref
953 # ideal_elyptical = []
954 # for i in 2:length(grid[2,1,:])
955 #     liftpsu = L0 * sqrt(1 - ((2 * grid[2,1,i])/(b))^2)
956 #     push!(ideal_elyptical, liftpsu)
957 # end

```



```

958
959
960 #plot!(xax, ideal_elyptical, label="Ideal Elyptical Wing",
    linewidth=3)
961 # pe1 = plot(xax, ideal_elyptical, ylims=(0, .55), label="Ideal
    Elliptical", linewidth=2, xlabel="Spanwise Location (m)",
    ylabel="Lift per Unit Span (N/m)")
962 # plot!(xax, filtered_Gamma, label="Actual Elliptical", linewidth
    =2)
963 # plot(pcc, pe1, layout=(2,1))
964
965 # plot(xax, filtered_Gamma, ylims=(0, 2.5), label="Ideal
    Elliptical", linewidth=2, xlabel="Spanwise Location (in)",
    ylabel="Lift (N)")
966
967 # println("Efficiency - Elyptical: ", CF[3]/CF[1]) #Lift to Drag
    Ratio
968 # AR = (bref^2)/Sref
969 # println((CF[3]^2)/(pi * CDiff * AR))
970 #println(CF[3])
971 #teapz(b_span, CL.1) (Find the integral of the lift distribution
    to get total lift)
972
973
974
975
976
977
978
979
980
981
982
983 # straight section of the beam
984 L_b1 = b_half/2 * 39.3701 # length of straight section of the
    beam in inches
985 r_b1 = [0, 0, 0] # starting point of straight section of the beam
986 nelem_b1 = n      2 # number of elements in the straight section of
    the beam
987 lengths_b1, xp_b1, xm_b1, Cab_b1 = discretize_beam(L_b1, r_b1,
    nelem_b1)
988
989 # swept section of the beam
990 L_b2 = b_half/2 * 39.3701 # length of swept section of the beam
991 r_b2 = [L_b1, 0, 0] # starting point of swept section of the beam

```

```

992 nelem_b2 = n      2 # number of elements in swept section of the
    beam
993 e1 = [1, 0, 0] # axis 1
994 e2 = [0, 1, 0] # axis 2
995 e3 = [0, 0, 1] # axis 3
996 frame_b2 = hcat(e1, e2, e3) # transformation matrix from local to
    body frame
997 lengths_b2, xp_b2, xm_b2, Cab_b2 = discretize_beam(L_b2, r_b2,
    nelem_b2;
998     frame = frame_b2)
999
1000
1001 # combine elements and points into one array
1002 nelem = nelem_b1 + nelem_b2 # total number of elements
1003 points = vcat(xp_b1, xp_b2[2:end]) # all points in our assembly
1004 start = 1:nelem_b1 + nelem_b2 # starting point of each beam
    element in our assembly
1005 stop = 2:nelem_b1 + nelem_b2 + 1 # ending point of each beam
    element in our assembly
1006 lengths = vcat(lengths_b1, lengths_b2) # length of each beam
    element in our assembly
1007 midpoints = vcat(xm_b1, xm_b2) # midpoint of each beam element in
    our assembly
1008 Cab = vcat(Cab_b1, Cab_b2) # transformation matrix from local to
    body frame
1009                                     # for each beam element in our assembly
1010
1011
1012
1013 # elem_lengths = lengths # already in inches
1014 elem_midT = [midpoints[i][1] for i in 1:nelem]
1015 q_liftT = [(filtered_Gamma[i] / lengths[i]) * 0.2242 for i in 1:
    nelem]
1016
1017
1018 # cross section
1019 h = 1 # inch
1020 w = 1.9685 # inch
1021
1022 # material properties
1023 E = 29000000 # lb/in^2
1024     = 0.28
1025     = 0.284 # lb sec^2/in^4
1026
1027 # shear and torsion correction factors

```

```

1028 ky = 0.83
1029 kz = 0.83
1030 kt = 1
1031
1032 A = h*w
1033 Iyy = w*h^3/12
1034 Izz = w^3*h/12
1035 J = Iyy + Izz
1036 # apply corrections
1037 Ay = A/ky
1038 Az = A/kz
1039 Jx = J/kt
1040
1041 G = E/(2*(1+ ))
1042 # println()
1043 # println(q_lift)
1044 # println()
1045 compliance = fill(Diagonal([1/(E*A), 1/(G*Ay), 1/(G*Az), 1/(G*Jx),
1046     1/(E*Iyy),
1047     1/(E*Izz)]), nelem)#creates a 6x6 compliance matrix for each
1048     element in the assembly, where the diagonal
1049     #entries correspond to the inverse of the stiffness of the
1050     element in each degree of freedom. The
1051     #compliance matrix is used to relate forces and moments to
1052     displacements and rotations in the beam
1053     #elements.
1054
1055 # println(compliance[1])
1056 for ww in 1:n
1057     local w = max(chord[ww], 0.01) * 39.3701 # avoid zero chord
1058     local A = h*w
1059     local Iyy = w*h^3/12
1060     local Izz = w^3*h/12
1061     local J = Iyy + Izz
1062     local Ay = A/ky
1063     local Az = A/kz
1064     local Jx = J/kt
1065     compliance[ww] = Diagonal([1/(E*A), 1/(G*Ay), 1/(G*Az), 1/(G*
1066         Jx), 1/(E*Iyy), 1/(E*Izz)])
1067 end
1068
1069 mass = fill(Diagonal([ *A, *A, *A, *J, *Iyy, *Izz]),
1070     nelem)#creates a 6x6 mass matrix for each element
1071 #in the assembly, where the diagonal entries correspond to the
1072 mass of the element in each degree of freedom.

```

```

1066 #The mass matrix is used to relate accelerations to forces and
      moments in the beam elements.
1067 total_mass_tapered = 0
1068 for ww in 1:n
1069     local w = max(chord[ww], 0.01) * 39.3701 # avoid zero chord
1070     local A = h*w
1071     local Iyy = w*h^3/12
1072     local Izz = w^3*h/12
1073     local J = Iyy + Izz
1074     local Ay = A/ky
1075     local Az = A/kz
1076     local Jx = J/kt
1077     mass[ww] = Diagonal([ *A, *A, *A, *J, *Iyy, *Izz])
1078     global total_mass_tapered += *A*lengths[ww] # mass per unit
      length * length of element
1079 end
1080 println("Total_Mass_for_Tapered_Wing:", total_mass_tapered)
1081
1082 assembly = Assembly(points, start, stop;
1083     compliance = compliance,
1084     mass = mass,
1085     frames = Cab,
1086     lengths = lengths,
1087     midpoints = midpoints)
1088
1089
1090 # mkpath("rotating-geometry")
1091 # write_vtk("rotating-geometry/rotating-geometry", assembly)
1092
1093 # ADDING POINT MASSES
1094 # m = 10 # mass
1095 # p = zeros(3) # relative location
1096 # J = zeros(3,3) # inertia matrix (about the point mass center of
      gravity)
1097
1098 # # create dictionary of point masses
1099 # point_masses = Dict(
1100 #     nelelem+1 => PointMass(m, p, J)
1101 # )
1102
1103 # DISTRIBUTED LOADS FROM AERODYNAMIC LIFT (using right-end value)
1104 distributed_loads = Dict{Int64, DistributedLoads{Float64}}{ }
1105 for ielem in 1:nelem
1106     local q = q_liftT[ielem] # lb/in, force per unit length for
      this element

```

```

1107     distributed_loads[ielem] = DistributedLoads(assembly, ielem;
1108         fz = (s) -> q    # constant load over the element
1109     )
1110 end
1111
1112
1113 # # DISTRIBUTED LOADS THAT ROTATE WITH THE BEAM
1114 # distributed_loads = Dict{Int64, DistributedLoads{Float64}}{ }()
1115 # for ielem in 1:nelem
1116 #     distributed_loads[ielem] = DistributedLoads(assembly, ielem;
1117 #         fz_follower = (s) -> 1)
1118 # end
1119
1120 # Defining PRESCRIBED CONDITIONS
1121 # create FIXED BOUNDARY CONDITION ON THE LEFT SIDE (ROOT) OF THE
    BEAM-->
1122 prescribed_conditions = Dict(
1123     # left support: fix all translations, block rotation about
        beam axis
1124     1 => PrescribedConditions(ux = 0, uy = 0, uz = 0, theta_x = 0,
        theta_y = 0, theta_z = 0)
1125 )
1126
1127 # PREALLOCATING MEMORY FOR OUR ANALYSIS(CAN USE StaticSystem,
    DynamicSystem, or ExpandedSystem(when using the mass matrix for
    differential equations))
1128 system = DynamicSystem(assembly)
1129
1130
1131
1132
1133
1134
1135
1136
1137 # Performing a Steady State ANALYSIS
1138 rpm = [0.0]
1139
1140 # linear_states = Vector{AssemblyState{Float64}}{ }(undef, length(rpm)
    )
1141 # for i = 1:length(rpm)
1142
1143 #     # global frame rotation
1144 #     w0 = [0, 0, rpm[i]*(2*pi)/60]
1145

```

```

1146 #         # perform linear steady state analysis
1147 #         _, state, converged = steady_state_analysis!(system,
assembly,
1148 #             angular_velocity = w0,
1149 #             prescribed_conditions = prescribed_conditions,
1150 #             distributed_loads = distributed_loads,
1151 #             linear = true)
1152
1153 #         # save result
1154 #         linear_states[i] = state
1155
1156 #     end
1157
1158 #     reset_state!(system)
1159
1160 nonlinear_states = Vector{AssemblyState{Float64}}(undef, length(
rpm))
1161 for i = 1:length(rpm)
1162
1163     # global frame rotation
1164     w0 = [0, 0, rpm[i]*(2*pi)/60]
1165
1166     # perform nonlinear steady state analysis
1167     _, state, converged = steady_state_analysis!(system, assembly,
angular_velocity = w0,
1168     distributed_loads = distributed_loads,
1169     prescribed_conditions = prescribed_conditions)
1170
1171
1172     # save result
1173     nonlinear_states[i] = state
1174
1175 end
1176
1177 # i = 1 # first RPM case
1178 # # vertical displacement u_z at every node
1179 # uz_all = [linear_states[i].points[n].u[3] for n in 1:length(
points)]
1180 # # x coordinates of each node
1181 # xvaluesnumerical = [points[n][1] for n in 1:length(points)]
1182 # println(uz_all)
1183
1184
1185 # #####LINEAR
1186 # i = 1 # for example, the 1st RPM case

```

```

1187 # uz_all = [linear_states[i].points[n].u[3] for n in 1:length(
      points)]
1188 # uy_all = [linear_states[i].points[n].u[2] for n in 1:length(
      points)]
1189 # ux_all = [linear_states[i].points[n].u[1] for n in 1:length(
      points)]
1190 # xvaluesnumerical = [points[aa][1] for aa in 1:length(points)]
1191 # thetaxGX = [linear_states[i].points[n].theta[2] for n in 1:
      length(points)]
1192 # #print(length(linear_states[i].elements))
1193 # #println(linear_states[i].elements[16].Mi)
1194 # moments = [linear_states[i].elements[n].Mi[2] for n in 1:length(
      linear_states[i].elements)]
1195 # xvalues12 = [midpoints[n][1] for n in 1:length(linear_states[i].
      elements)]
1196 # shears = [linear_states[i].elements[n].Fi[3] for n in 1:length(
      linear_states[i].elements)]
1197
1198 #####NONLINEAR
1199 i = 1 # for example, the 1st RPM case
1200 uz_allT = [nonlinear_states[i].points[n].u[3] for n in 1:length(
      points)]
1201 uy_allT = [nonlinear_states[i].points[n].u[2] for n in 1:length(
      points)]
1202 ux_allT = [nonlinear_states[i].points[n].u[1] for n in 1:length(
      points)]
1203 xvaluesnumericalT = [points[aa][1] for aa in 1:length(points)]
1204 thetaxGXT = [nonlinear_states[i].points[n].theta[2] for n in 1:
      length(points)]
1205 #print(length(linear_states[i].elements))
1206 #println(linear_states[i].elements[16].Mi)
1207 momentsT = [nonlinear_states[i].elements[n].Mi[2] for n in 1:
      length(nonlinear_states[i].elements)]
1208 xvalues12T = [midpoints[n][1] for n in 1:length(nonlinear_states[i]
      ).elements)]
1209 shearsT = [nonlinear_states[i].elements[n].Fi[3] for n in 1:length
      (nonlinear_states[i].elements)]
1210 println("Total_Drag_Coeffiecient_for_Tapered_Wing:", CDiff)
1211 println()
1212
1213
1214
1215
1216
1217

```

```

1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234 # PLOTTING THE RESULTS
1235 pwxGXz = plot(xvaluesnumerical, uz_all, label="Elyptical_Wing",
               xlabel="Distance_along_the_beam(in)", ylabel="Vertical
               Displacement(in)", title="Vertical_Displacement_W(y)")
1236 plot!(xvaluesnumericalR, uz_allR, label="Rectangular_Wing", xlabel=
               "Distance_along_the_beam(in)", ylabel="Vertical_Displacement_
               (in)", title="Vertical_Displacement_W(y)")
1237 plot!(xvaluesnumericalT, uz_allT, label="Tapered_Wing", xlabel="
               Distance_along_the_beam(in)", ylabel="Vertical_Displacement_
               (in)", title="Vertical_Displacement_W(y)")
1238
1239
1240 pthetaxGX = plot(xvaluesnumerical, thetaxGX, label="Elyptical_Wing
               ", xlabel="Distance_along_the_beam(in)", ylabel="Rotation_
               ", title="Rotation_ (y)")
1241 plot!(xvaluesnumericalR, thetaxGXR, label="Rectangular_Wing",
               xlabel="Distance_along_the_beam(in)", ylabel="Rotation_
               ( )", title="Rotation_ (y)")
1242 plot!(xvaluesnumericalT, thetaxGXT, label="Tapered_Wing", xlabel="
               Distance_along_the_beam(in)", ylabel="Rotation_
               ( )", title="
               Rotation_ (y)")
1243
1244
1245 pMxGX = plot(xvalues12, moments, label="Elyptical_Wing", xlabel="
               Distance_along_the_beam(in)", ylabel="Moment_(lb-in)", title="
               Moment_M(y)")
1246 plot!(xvalues12R, momentsR, label="Rectangular_Wing", xlabel="
               Distance_along_the_beam(in)", ylabel="Moment_(lb-in)", title="
               Moment_M(y)")

```



```

1247 plot!(xvalues12T, momentsT, label="Tapered_Wing", xlabel="Distance
      _along_the_beam(in)", ylabel="Moment_(lb-in)", title="Moment_M
      (y)")
1248
1249 pVxGX = plot(xvalues12, shears, label="Elyptical_Wing", xlabel="
      Distance_along_the_beam(in)", ylabel="Shear_Force_(lb)", title
      ="Shear_Force_V(y)")
1250 plot!(xvalues12R, shearsR, label="Rectangular_Wing", xlabel="
      Distance_along_the_beam(in)", ylabel="Shear_Force_(lb)", title
      ="Shear_Force_V(y)")
1251 plot!(xvalues12T, shearsT, label="Tapered_Wing", xlabel="Distance_
      along_the_beam(in)", ylabel="Shear_Force_(lb)", title="Shear_
      Force_V(y)")
1252
1253 lift_plot = plot(elem_mid, q_lift, label="Ideal_Elliptical",
      linewidth=2, xlabel="Spanwise_Location(in)", ylabel="Lift_(lb)
      ")
1254 plot!(elem_midR, q_liftR, label="Rectangular", linewidth=2, xlabel
      ="Spanwise_Location(in)", ylabel="Lift_(lb)", ylims=(0, 8))
1255 plot!(elem_midT, q_lift_T, label="Tapered", linewidth=2, xlabel="
      Spanwise_Location(in)", ylabel="Lift_(lb)", ylims=(0, 8))
1256
1257 plot(pwxGXz, pthetaxGX, pMxGX, pVxGX, lift_plot, layout=(3, 2),
      size=(1500,1500), margin=5mm)
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278

```

1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321

```
1322 # plot(elem_midR, q_liftR, label="Distributed Lift", xlabel="
      Distance along the beam (in)", ylabel="Lift per Unit Length (N/
```

```
in)", title="Distributed Lift Along the Beam")
```

1323

1324

1325

1326

1327

1328

1329

1330

1331

1332

1333

1334

1335

1336

1337

1338

1339

1340

1341

1342

1343

1344

1345

1346

1347

1348

1349

1350

1351

1352

1353

1354

1355

1356

1357

1358

1359

1360

```
1361 # # # #FINDING THE EFFECT THE SPAN LENGTH OF THE WING HAS ON THE
```

```
1362 # # # #MOMENT AT IT'S ROOT
```

```
1363 # # # using FLOWMath
```

```
1364 # # # using Plots
```

```
1365 # # # moments_analytical = []
```

```
1366 # # # x_values_ma = []
```

```

1367 # # # density = 0.38
1368 # # # Vinf = 10
1369 # # # for b in range(1, 10, length=10)
1370 # # #     push!(x_values_ma, b)
1371 # # #     moment_wing = []
1372 # # #     for y in range(0, b/2, length=100)
1373 # # #         c = 1 * sqrt(1 - (2*y/(b))^2)
1374 # # #         lftpus = density * Vinf * c # converting circlation
            to lift per unit span
1375 # # #         added_moment = lftpus * y
1376 # # #         push!(moment_wing, added_moment)
1377 # # #     end
1378 # # #     total_moment = sum(moment_wing)
1379 # # #     push!(moments_analytical, total_moment)
1380 # # # end
1381 # # # plot(x_values_ma, moments_analytical, xlabel="Span length (m
            )", ylabel="Moment (N-m)",
1382 # # # title="Moments at the Root of Wings with Varying Span
            Lengths",
1383 # # # size=(650, 400), legend=false)
1384
1385 # qliftplot = plot(elem_mid, q_lift, label="Distributed Lift,
            Elliptical", xlabel="Distance along the beam (in)", ylabel="
            Lift per Unit Length (N/in)", title="Distributed Lift Along the
            Beam")

```